Index

LIST OF FIGURES

LIST OF TABLES

1. Introduction and Problem Statement

In every actual event like a meal for a wedding, seminar for a corporation or a festival for a college, person must coordinate the venue, the guests, the timeline and the final invoice. To manage many venues at the same time, an organization encounters problems that often happen. It is difficult to track which hall is open for use and to record the identity of the person who reserved at the specific time. As double bookings can occur, the organization must work to prevent them. By using paper registers or static spreadsheets, people make mistakes because those methods exist only through manual updates.

This project addresses that exact scenario by building a working software model of a event management system using C++. The system stores event's slot as nodes in a dynamically allocated linked list, sorts them automatically whenever a new slot is added, allows customers to book available slots and enter their details, and generates a computed invoice at checkout. All interactions happen through a clean, numbered text menu that runs in the terminal.

1.1 Why this problem is worth solving as a code

The problem is well-suited for a data structures project because it naturally involves creation and deletion of records at runtime, a need for ordered access , and repeated traversal to find specific records. These are precisely the scenarios where a linked list outperforms a static array. If a programmer uses a static array, they must state the total number of locations before the program starts but in a real environment, the number of venues increases or decreases. By using a linked list the structure can change size when the data is changed.

1.2 Programming Language Choice – C++

C++ is chosen because it supports two worlds simultaneously the object-oriented which includes classes, encapsulation, methods and the systems-level which includes raw pointers, manual memory allocation with new and delete, direct control over what lives on the heap vs. the stack. The project exploits both. The classes User, Event Node, and Event Manager use OOP's design, while the linked list nodes are created and managed using raw pointers which explains understanding of dynamic data structures work under the hood.

1.3 Scope of the Point

The system performs multiple functions when a user adds a new venue slot they enter how many people it holds and how long the event lasts. To see which locations are open a user views all venues that do not have a booking. If a person wants to reserve a space, the system books a slot after the user enters the details for the customer. For situations where a user must find a record, they search for the booking using the first name of the client. As a customer checks out, the system creates an itemized invoice that lists the cost for staff, the cost for food and the cost for overhead. To understand the current status of the business, a user views a summary of every active booking in every venue. And the program closes when the user chooses to exit.

2. System Architecture and Design Overview

There are 3 types of Data Layer which can be stated as follows:-

Layer	What it Contains	Purpose
Data Layer	User class, Event Node, global head pointer	Defines how information is stored and linked in memory
Algorithm Layer	Split list(), Merge Sorted List(), Merge Sort()	Keeps the event list sorted by Event ID at all times
Application Layer	Event Manager Class, Manage Events Menu(), Main()	Handles all user interactions and business logic

Table No.1

2.1 Program Flow from Start to Exit

When the program runs, the global pointer head is seted to null ptr, meaning the linked list is empty. The main() function immediately enters in do-while loop and draws the main menu. The user picks an option between 1 and 7. There is a guard condition built into main() that checks whether head is null ptr before allowing options 2 through 6 — this prevents operations on an empty list.

Option 1 leads to a sub-menu (manage Events Menu) where the administrator can add new venue slots. Each new venue creates a node on the heap, inserts it at the front of the list, and then triggers Merge Sort to restore sorted order.

Options 2 through 6 each call the corresponding Event Manager method. Option 7 exits the do-while loop, reaches return 0, and ends the program.

gram.

2.2 Memory Model

Every Event Node is created with new Event Node(), which places it in the heap in which a region of memory that persists for the duration of the program. The global pointer head keeps track of the list's starting point. When an event is booked, no new memory is allocated — the existing node's is Booked flag is set to true and the client data fields are filled in. In generate Bill(), after the invoice is printed, the node's is Booked flag is reset to false, making the venue available again. Note that in the current version, the node itself is not deleted — it remains in the list for future use.

2.3 Data Flow Diagram (Textual)

User Input ⟶ main() switch-case ⟶ Event Manager method ⟶ Linked List traversal ⟶ Output to console

New event added ⟶ Node created on heap ⟶ Inserted at head ⟶ mergeSort() sorts list by eventID

Bill generated ⟶ Node found ⟶ Bill computed ⟶ isBooked reset to false ⟶ Venue freed

3. Header Files and Preprocessor Directives

The first five lines of the program pull in the libraries that the rest of the code depends on. Each one has a specific purpose, and none of them are included arbitrarily.

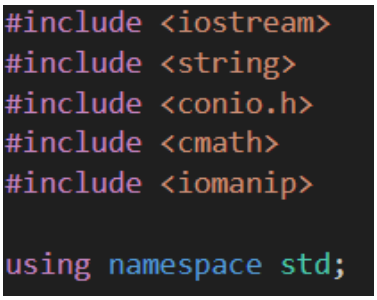

Figure No.1

Header / Directive	What It Provides	Where Used in This Program
#include <iostream>	cin, cout — standard input and output	Every function that reads input or prints output
#include <string>	The string data type and its operations	User class fields (name, address, phone, date, venueName)
#include <conio.h>	getch() — waits for a keypress without Enter	Used after every screen display as a pause mechanism
#include <cmath>	ceil() — ceiling of a decimal number	generateBill() — rounds up the number of staff required
#include <iomanip>	setprecision(), fixed — formatted decimal output	generateBill() — prints currency values to 2 decimal places
using namespace std;	Removes the need to write std:: before standard names	Applied globally — simplifies all cout, cin, string usage

Table No.2

The use of <conio.h> is worth noting specifically. This header is not part of the C++ standard — it is a Windows-specific extension provided by compilers like MinGW and Turbo C++. The getch() function it provides reads a single character from the keyboard without printing it and without requiring the user to press Enter. In a console application, this is the standard way to pause the screen after displaying output so the user can read it before the menu redraws.

The <cmath> header is included specifically for the ceil() function used in billing. Dividing the number of guests by 10 using integer arithmetic would always round down — but staff count should always round up (you cannot hire half a staff member). ceil(actualGuests / 10.0) handles this correctly.

4. The User Class

The program defines the User class before any other classes - it is an organizational tool because it lacks methods or logic. To achieve its purpose, the class groups the personal and financial details of a customer into one unit with a name. By doing this the program is able to store those details together in a single field within an EventNode.

```
class User {
public:
    string name;
    string address;
    string phone;
    string date;
    int bookingID;
    float advancePayment;
};
```

Figure No.2

4.1 Field by Field Explanation

Field Name	Data Type	What It Stores
name	string	The first name of the client making the booking
address	string	The client's city (entered as a single word during booking)
phone	string	Contact phone number — stored as a string to handle leading zeros
date	string	The scheduled date of the event, entered by the client
bookingID	int	A unique integer identifier assigned by the admin at booking time
advancePayment	float	The deposit already paid by the client, deducted from the final bill

Table No.3

4.2 Phone Number as String – Design Rationale

In the system the developer stores the phone number as a string instead of an integer. It is intentional to use this format because phone numbers can include a zero at the beginning. In case the system employs an integer, then these leading zeros will be stripped off automatically. Since the digits will be stored as strings, then the database will store these numbers exactly as input by the user. In case these numbers are excessively large, then storing them as strings would ensure that they do not exceed the storage capacity of the data type

employed. This is why this technique is often used by most applications. Nearly all databases store phone numbers as strings.

5. The Event Class Node

The Event Node class is the structure of entire program. Each venue slot in the system is represented by one Event Node object. These objects are chained together through pointers to form the singly linked list that the program will use.

```
class EventNode {
public:
    int eventID;
    int timeDuration;
    int guestCount;
    string venueName;
    bool isBooked;

    User client;
    EventNode* next;

    EventNode() {
        next = nullptr;
        isBooked = false;
    }
};
```

Figure No.3

5.1 Field by Field Explanation

Field	Type	Purpose
eventID	int	Unique number identifying the event slot. Used as the sort key for Merge Sort
timeDuration	int	Maximum time limit for the event, stored in minutes
guestCount	int	Maximum number of guests the venue can accommodate
venueName	string	Name of the venue (e.g., GrandHall, RoseGarden). Entered as one word
isBooked	bool	Tracks whether this slot is currently occupied. true = taken, false = available

client	User	A nested User object — stores all customer data when the event is booked
next	EventNode*	Pointer to the next node in the linked list. nullptr if this is the last node

Table No.4

5.2 The Default Constructor

There exists only one constructor in the class called EventNode(). This constructor automatically fires when creating a new EventNode using the command new EventNode(). There exist only two specific activities done by this constructor. It first sets the value of the next pointer to nullptr. This implies that the newly created node cannot point to any other address that may have random information within it. It also initializes the value of the isBooked boolean field to false. This means that all venues will be available until someone else makes a booking at a later time. In case this constructor was not there, then these fields would have contained random information.

5.3 The Nested User Object

One of the more nice design choices in this program is by fixing a User object directly inside EventNode rather than maintaining a separate array or list of customers. This means each node is completely self-contained you don't need to look up at a separate table to find who booked the venue. When bookEvent() fills in name of client, client's phone, and so on, that data lives right inside the same memory block as the event details. This makes retrieval in searchUserByName() and generateBill() simple: find the node, and the client data is shown.

6. The Global Head Pointer

```
EventNode* head = nullptr;
```

Figure No.4

This single line is what makes the linked list work. head is a pointer to an EventNode — specifically to the first node in the list. It is declared globally, meaning every function in the program can access and modify it without needing to pass it as an argument.

Initially, head is set to nullptr. This indicates an empty list. As events are added, head is updated to point to the front of the sorted list. When the list has five nodes, for example, head points to the node with the smallest Event ID, which points to the next, which points to the next, and so on, until the final node's next is nullptr.

State	head value	What It Means

Program just started	nullptr	No events have been added yet
One event added (ID=3)	→ Node(ID=3)	Single node; next is nullptr
Three events added (IDs 5,2,8, sorted)	→ Node(ID=2) → Node(ID=5) → Node(ID=8) → nullptr	List sorted by ID after each insert
All events billed / freed	head still → nodes	Nodes remain; isBooked reset to false

Table No.5

Using a global pointer is a common technique in student-level C++ linked list programs. A more advanced approach would pass the head pointer as a parameter to each function (or better, encapsulate it inside a class). In this program, the global approach keeps the code concise and clear.

7. Merge Sort on the Linked List

Merge Sort is a sorting algorithm. It is applied to the linked list each time when a event is added. This implementation consists of three functions that work together: splitList(), mergeSortedLists(), and mergeSort().

7.1 Why Merge Sort Instead of Bubble Sort or Selection Sort

Bubble Sort and Selection Sort are conceptually simple to understand. However, it result $O(n^2)$ time which means 100 elements required 10,000 comparisons. Both algorithms are designed around arrays, where you can access the element at position i directly. In a linked list, you cannot do that. For reaching 50th element, you have to traverse 49 pointers. Array-style sorting to a linked list introduces unnecessary complexity, merge Sort was designed with divide-and-conquer, not direct indexing. It splits the list and sorts each half independently, and merges them back. This is a natural fit for linked lists, and which results O(n log n) performance even in the worst case.

7.2 Function 1 — splitList()

```
void splitList(EventNode* source, EventNode** frontRef, EventNode** backRef) {
    EventNode* fast = source->next;
    EventNode* slow = source;

    while (fast != nullptr) {
        fast = fast->next;
        if (fast != nullptr) {
            slow = slow->next;
            fast = fast->next;
        }
    }
    *frontRef = source;
    *backRef = slow->next;
    slow->next = nullptr;
}
```

Figure No.5

splitList() takes the linked list starting at source and divides it into two independent halves. The result is written into *frontRef (the left/first half) and *backRef (the right/second half). The function uses the classic fast-slow pointer technique to find the midpoint without needing to know the length of the list.

How the Fast-Slow Pointer Technique Works

Two pointers are initialized: fast starts at source->next (one step ahead), and slow starts at source. The loop runs while fast is not nullptr. Each iteration: first, fast advances one step. Then, if fast is still not nullptr, slow also advances one step AND fast advances one more step. This means fast moves two nodes forward for every one node slow moves.

Because fast moves at double speed, by the time fast reaches the end of the list, slow is at the midpoint. The list is then cut by setting slow->next = nullptr, which disconnects the second half from the first. The pointer to the start of the second half (slow->next before it was set to nullptr) is stored in *backRef.

List State	fast position	slow position	Action
Start: 4 nodes A→B→C→D	B	A	Initialize
Iteration 1	C	A (fast!=null, so slow→B)	fast=C, then slow=B, fast=D
Iteration 2	nullptr	B	fast→null, stop. Cut at slow=B
Result	frontRef = A→B→null	backRef = C→D→null	Split complete

Table No.6

7.3 Function 2 — mergeSortedLists()

```
EventNode* mergeSortedLists(EventNode* leftHalf, EventNode* rightHalf) {
    if (leftHalf == nullptr) return rightHalf;
    if (rightHalf == nullptr) return leftHalf;

    EventNode* result = nullptr;

    if (leftHalf->eventID <= rightHalf->eventID) {
        result = leftHalf;
        result->next = mergeSortedLists(leftHalf->next, rightHalf);
    } else {
        result = rightHalf;
        result->next = mergeSortedLists(leftHalf, rightHalf->next);
    }
    return result;
}
```

Figure No.6

This function takes two already-sorted linked lists and merges them into one sorted list. It is called after both halves have been recursively sorted by mergeSort().

The logic is recursive process. At each call, it compares the eventID of the front node of each half, whichever is smaller becomes the next node in the output list. The half pointer advances one by one (leftHalf->next or rightHalf->next), and the function calls itself recursively with the updated pointers. When one half becomes nullptr the other half is attached directly which is already sorted.

Worked Example — Merging {2,5} with {3,8}

Call	leftHalf front	rightHalf front	Decision	Result so far
1st	2	3	2 < 3 → pick 2	2 → ...
2nd	5	3	3 < 5 → pick 3	2 → 3 → ...
3rd	5	8	5 < 8 → pick 5	2 → 3 → 5 → ...
4th	null	8	leftHalf empty → append 8	2 → 3 → 5 → 8

Table No.7

7.4 Function 3 — mergeSort()

```
void mergeSort(EventNode** headRef) {
    EventNode* currentHead = *headRef;
    EventNode* leftHalf;
    EventNode* rightHalf;

    if ((currentHead == nullptr) || (currentHead->next == nullptr)) {
        return;
    }

    splitList(currentHead, &leftHalf, &rightHalf);
    mergeSort(&leftHalf);
    mergeSort(&rightHalf);
    *headRef = mergeSortedLists(leftHalf, rightHalf);
}
```

Figure No.7

mergeSort() is the starting point. It receives a pointer-to-pointer to head (written as EventNode** headRef) so it can update the actual head variable rather than just a local copy. The base case handles two cases: if the list is empty (nullptr) or it has only one node (next == nullptr) it is already sorted.

The splitList() to splits the list, recursively calls itself on each half which will keeps splitting until single nodes is reached, and then calls mergeSortedLists() to combine the sorted halves. The return value of mergeSortedLists() head of the meged sorted list is written back through *headRef, updating the real head pointer in the caller.

This function is called from addEvent() as mergeSort(&head) every time a new node is added ensuring the list is always sorted.

8. The EventManager Class Declaration

```
class EventManager {
public:
    void addEvent(int id);
    void displaySingleEvent(EventNode* event);
    void viewAvailableEvents();
    void bookEvent();
    void searchUserByName(string searchName);
    void generateBill(int id);
    void viewGuestSummary();
};
```

Figure No.8

The Event Manager class acts as the central controller for all application operations. It declares seven methods — each corresponding to one user-facing feature to the system. The class itself holds no data members. Instead, all methods operate on the global linked list through the head pointer.

The separation between the data structure (EventNode linked list) and the operations on that data (EventManager) is a practical demonstration of the OOP's principle. If you ever wanted to change how events are stored internally from a linked list to a binary search tree you would only modify the method bodies inside Event Manager. The class declaration and its interface to the rest of the program would remain identical.

Method	Parameter	Returns	Purpose
addEvent	int id	void	Creates a new EventNode, inserts at head, triggers sort
displaySingleEvent	EventNode* event	void	Prints the four key fields of one event node
viewAvailableEvents	(none)	void	Displays all nodes where isBooked == false
bookEvent	(none)	void	Finds an event by ID and fills in the client details
searchUserByName	string searchName	void	Finds and displays booking records matching a name
generateBill	int id	void	Computes and prints the final invoice for a booked event
viewGuestSummary	(none)	void	Lists all nodes where isBooked == true

Table No. 8

9. addEvent() — Creating a New Venue Slot

```
void EventManager::addEvent(int id) {
    EventNode* newEvent = new EventNode();
    newEvent->eventID = id;

    cout << "\n    Total Guest Capacity : ";
    cin >> newEvent->guestCount;
    cout << "\n    Time Limit (in minutes) : ";
    cin >> newEvent->timeDuration;
    cout << "\n    Venue Name (One word, e.g., GrandHall) : ";
    cin >> newEvent->venueName;
```

Figure No.9

9.1 Step-by-Step Walkthrough

The function receives the eventID as a parameter — it was already read and validated (for uniqueness) by manageEventsMenu() before this function was called. The first action is new EventNode(), which allocates memory for the node on the heap and runs the constructor, setting next to nullptr and isBooked to false.

The three cin statements populate guestCount, timeDuration, and venueName. These are read directly from the keyboard and stored in the new node's fields. Note that venueName uses cin (not getline), which means it reads only until a space — this is why the prompt instructs the user to enter the name as one word.

9.2 Head Insertion Logic

```
newEvent->next = head;   // new node points to what was previously the first node
head = newEvent;         // head now points to the new node — it becomes the new front
```

This is the standard technique for inserting at the head of a singly linked list. The new node's next pointer is set to the current head before head is updated. If this order were reversed, if head was updated first the reference to the old first node will be lost permanently. The two-line order ensures the chain is never broken.

9.3 Sorting After Insertion

After head insertion, the list is no longer sorted, the new node sits at the front regardless of its ID. mergeSort is called immediately to restore sorted order. The &head syntax passes the address of the global head pointer, allowing mergeSort() to update head to point to the new front of the sorted list.

The practical effect: no matter what order events are added, the list is always sorted by Event ID by the time any other operation runs.

10. displaySingleEvent() — Printing One Record

```
void EventManager::displaySingleEvent(EventNode* event) {
    cout << "\n   Event ID     : " << event->eventID;
    cout << "\n   Venue Name   : " << event->venueName;
    cout << "\n   Max Guests   : " << event->guestCount;
    cout << "\n   Time Limit   : " << event->timeDuration << " minutes\n";
}
```

Figure No.10

This is a utility function. It is meant to save repetition by not having to repeat four cout statements wherever required. Instead of including the four-line code block for printing wherever necessary like in viewAvailableEvents(), the call is made to displaySingleEvent() function along with passing the node pointer.

The argument passed to this function is of type EventNode* event. -> operator is utilized here because event is a pointer type variable and the -> is used when accessing fields using pointer variable in C++. The output contains Event Id, Venue Name, max number of guests, and duration limit.

This approach highlights an important concept from software engineering. If there comes a need to make changes in the display format, the changes can simply be made in one location.

11. viewAvailableEvents() -- Browsing Free Venues

```
void EventManager::viewAvailableEvents() {
    EventNode* temp = head;
    bool foundAny = false;

    cout << "\n\t\t\t\t--- Available Venues ---\n";
    while (temp != nullptr) {
        if (!temp->isBooked) {
            displaySingleEvent(temp);
            cout << "   ---------------------------------";
            foundAny = true;
        }
        temp = temp->next;
    }

    if (!foundAny) {
        cout << "\n   Sorry, all venues are currently booked or none exist.";
    }
    cout << "\n\n   Press any key to return...";
    getch();
}
```

Figure No.11

11.1 Traversal Logic Explained

A local pointer temp is initialized to head. This is the standard linked list traversal pattern: never move the actual head pointer, because losing head means losing access to the entire list. Temp is a disposable copy in which the while loop runs as long as temp is not nullptr, when temp becomes nullptr, it means the last node's next has reached the end.

Inside the loop, the condition !temp->isBooked filters for available venues. If the node is not booked, displaySingleEvent() prints its details and a separator line is drawn out. The boolean foundAny tracks wheather at least one available venue is found. If the loop ends and foundAny is false, a message informs the user that all venues are occupied.

11.2 Why temp = temp->next Advances the Loop

Each node has a next pointer that stores the address of the following node. The statement temp = temp->next makes temp point to that next node, effectively moving one step forward in the list. This is equivalent to incrementing an array index — it is how you move through a linked list.

12. bookEvent() — Customer Booking Workflow

```
void EventManager::bookEvent() {
    int id;
    cout << "\n   Enter Event ID to book : ";
    cin >> id;

    EventNode* temp = head;
    while (temp != nullptr) {
        if (temp->eventID == id) {
            if (temp->isBooked) {
                cout << "\n   Sorry, this Event ID is already booked.";
                getch();
                return;
            }

            system("cls");
            cout << "\n\t\t\t\t--- Event Booking Form ---\n\n";
            cout << "   Enter Booking ID Number : "; cin >> temp->client.bookingID;
            cout << "   Enter Client First Name : "; cin >> temp->client.name;
            cout << "   Enter City             : "; cin >> temp->client.address;
            cout << "   Enter Phone Number     : "; cin >> temp->client.phone;
            cout << "   Enter Event Date       : "; cin >> temp->client.date;
            cout << "   Enter Advance Payment  : $"; cin >> temp->client.advancePayment;

            temp->isBooked = true;
            cout << "\n\n   Venue Booked Successfully for " << temp->client.name << "!";
            getch();
            return;
        }
        temp = temp->next;
    }
    cout << "\n   Event ID not found.";
    getch();
}
```

Figure No.12

12.1 Search-Then-Fill Pattern

bookEvent() begins by asking the user for an Event ID to book. It then traverses the linked list to find a node with a matching eventID. In the linear search, the list is checked node by node from head to tail. In the list of n nodes, this takes at most n comparisons.

When a matching node is found the function checks isBooked before moving forward. If the venue is found booked, the function prints out an error message showing no booking form is found. It avoids deeply nested if-else blocks by exiting as soon as an error condition is detected.

12.2 Filling Data Directly Into the Node

Notice that client data is written directly into temp->client.bookingID, temp->client.name, and so on. Because temp points to the actual node in the heap (not a copy), these assignments modify the real node's data. The dot notation .client accesses the nested User object inside the EventNode, and .name (for example) accesses the field within it.

This is a critical property of pointers: operations through a pointer affect the original data. If temp were a copy of the node rather than a pointer to it, the assignments would be lost when the function returned.

12.3 Setting isBooked = true

After all six customer fields are filled, temp->isBooked = true marks the venue as occupied. From this point on, viewAvailableEvents() will skip this node, and any attempt to book it again through bookEvent() will trigger the already-booked error message. Only generateBill() can reset isBooked to false — which happens after the final invoice is paid.

13. searchUserByName() — Finding a Client Record

```
void EventManager::searchUserByName(string searchName) {
    EventNode* temp = head;
    bool found = false;

    while (temp != nullptr) {
        if (temp->isBooked && temp->client.name == searchName) {
            system("cls");
            cout << "\n\t\t\t\t--- Client Booking Record ---\n";
            cout << "\n   Client Name  : " << temp->client.name;
            cout << "\n   Event ID     : " << temp->eventID;
            cout << "\n   Venue        : " << temp->venueName;
            cout << "\n   Date         : " << temp->client.date;
            cout << "\n\n   Press any key to continue...";
            found = true;
            getch();
        }
        temp = temp->next;
    }
    if (!found) {
        cout << "\n   No active bookings found for client: " << searchName;
        getch();
    }
}
```

Figure No.13

13.1 Search Condition — Double Guard

The if condition inside the loop checks two things simultaneously using the && (AND) operator: temp->isBooked must be true AND temp->client.name must equal searchName. The isBooked check is important because unbooked nodes have an empty client.name field, which could accidentally match an empty search string. By requiring isBooked first, the function only looks at nodes that genuinely have customer data.

13.2 String Comparison with ==

The == operator on C++ strings performs a full character-by-character comparison. This search is case-sensitive and exact — searching for "harsh" will not find a record stored as "Harsh". The function does not break out of the loop after finding a match; it continues traversing. This means if the same client name appears in multiple bookings which should not happen with unique booking IDs, but if possible with the current implementation, all matching records will be printed.

13.3 The found Flag Pattern

The boolean found variable starts as false. If any match is found, it is set to true. After the loop ends, if found is still false, the not-found message is printed. This is a clean way to distinguish between zero results and one-or-more results without counting matches.

14. generateBill() — Invoice Computation and Checkout

```
void EventManager::generateBill(int id) {
    EventNode* temp = head;

    while (temp != nullptr) {
        if (temp->isBooked && temp->eventID == id) {
            system("cls");
            int actualGuests, actualTime;

            cout << "\n\t\t\t\t--- Final Billing Checkout ---\n";
            cout << "\n   Enter Actual Number of Guests : "; cin >> actualGuests;
            cout << "\n   Enter Actual Time (Minutes)   : "; cin >> actualTime;

            double staffNeeded = ceil(actualGuests / 10.0);
            double staffCostPerHour = 30.00;
            double staffCostPerMin = 0.50;
            double costPerMeal = 100.00;
            double fixedOverhead = 2500.00;

            double totalStaffCost = ((actualTime / 60) * staffCostPerHour) + ((actualTime % 60) * staffCostPerMin);
            totalStaffCost *= staffNeeded;

            double totalFoodCost = actualGuests * costPerMeal;
            double totalBill = totalFoodCost + totalStaffCost + fixedOverhead;

            system("cls");
            cout << "\n\t\t\t\t--- Final Invoice ---\n";
            cout << "\n   Client Name      : " << temp->client.name;
            cout << "\n   Event ID         : " << temp->eventID;
            cout << "\n   ----------------------------------------";
            cout << fixed << setprecision(2);
            cout << "\n   Food Cost        : $" << totalFoodCost;
            cout << "\n   Staff Cost       : $" << totalStaffCost << " (" << staffNeeded << " staff members)";
            cout << "\n   Fixed Overhead   : $" << fixedOverhead;
            cout << "\n   ----------------------------------------";
            cout << "\n   Grand Total      : $" << totalBill;
            cout << "\n   Advance Paid     : $" << temp->client.advancePayment;
            cout << "\n   Remaining Due    : $" << totalBill - temp->client.advancePayment;
            cout << "\n   ----------------------------------------\n";

            int accNum;
            cout << "\n   Enter Bank Account Number to pay remaining balance: ";
            cin >> accNum;
            cout << "   Processing Payment... Transaction Successful!";

            temp->isBooked = false;
            cout << "\n\n   Event checked out. Venue is now available again.";
            getch();
            return;
        }
        temp = temp->next;
    }
    cout << "\n   Event ID not found or is not currently booked.";
    getch();
}
```

Figure No.14

14.1 Actual vs. Booked Figures

At checkout, not at booking time, the function asks for actualGuests and actualTime. This is an important design choice as a person might book a place for 100 people for 3 hours, but only 85 people show up and the event ends in 2 hours and 20 minutes. Charging based on real numbers is fair and correct. The guestCount and timeDuration stored in the node are not billing figures they are limits on how many people can attend an event.

14.2 The Billing Formula — Line by Line

14.2.1 Staff Count:

```
double staffNeeded = ceil(actualGuests / 10.0);
```

One staff member is required per 10 guests. The division uses 10.0 a double, instead of 10 an int, to force floating-point division. Without this, integer division shortens like 25/10 would give 2 instead of 2.5. ceil() rounds up to 3 because you cannot hire a fraction of a person as a staff.

14.2.2 Staff Cost Calculation:

```
double totalStaffCost = ((actualTime / 60) * staffCostPerHour)
                      + ((actualTime % 60) * staffCostPerMin);
totalStaffCost *= staffNeeded;
```

The time in minutes is split into full hours & minutes. Full hours are billed at $30.00 per staff member/hr. Remaining minutes are billed at $0.50 per staff member/min. The total is multiplied by the number of staff members required.

14.2.3 Food Cost:

```
double totalFoodCost = actualGuests * costPerMeal;
```

Each guest is charged $100.00 for the meal package. This is a fair per-head rate applied to the actual number of guests present.

14.2.4 Grand Total:

```
double totalBill = totalFoodCost + totalStaffCost + fixedOverhead;
```

The three components food cost, staff cost, and a fixed charge of $2,500.00 covering administrative, logistics, and facility costs are summed to produce the grand total.

14.3 Worked Billing Example

Suppose the actual guest count is 25 and the actual time is 90 minutes. The calculations proceed as follows:

Component	Calculation	Amount
Staff needed	ceil(25 / 10.0) = ceil(2.5)	3 staff members
Staff cost (full hours)	(90 / 60) × $30.00 = 1 × $30.00	$30.00 per staff

Staff cost (extra mins)	(90 % 60) × $0.50 = 30 × $0.50	$15.00 per staff
Total staff cost	($30.00 + $15.00) × 3 staff	$135.00
Food cost	25 guests × $100.00	$2,500.00
Fixed overhead	(constant)	$2,500.00
Grand Total	$135.00 + $2,500.00 + $2,500.00	$5,135.00
Less: Advance paid	(e.g., $1,000 paid at booking)	−$1,000.00
Remaining Due		$4,135.00

Table No.9

14.4 Payment and Venue ReleasePayment and Venue Release

After the invoice is printed, the function asks for a bank account number to process. This situational as there is no actual transaction taking place. A confirmation message is printed, and then temp->isBooked is set to false. This releases the venue, it will now appear again in viewAvailableEvents() and can be booked by another customer. The node remains in the linked list, only its booking status is reset.

15. viewGuestSummary() — Active Bookings Dashboard

```
void EventManager::viewGuestSummary() {
    system("cls");
    cout << "\n\t\t\t\t\t--- All Active Bookings ---\n\n";

    if (head == nullptr) {
        cout << "   System is empty. No venues exist yet.";
    } else {
        EventNode* temp = head;
        bool foundAny = false;

        while (temp != nullptr) {
            if (temp->isBooked) {
                cout << "   Client: " << temp->client.name << " | Event ID: " << temp->eventID << " | Date: " << temp->client.date << "\n";
                foundAny = true;
            }
            temp = temp->next;
        }

        if (!foundAny) cout << "   No active bookings at the moment.";
    }
    cout << "\n\n   Press any key to return...";
    getch();
}
```

Figure No.15

viewGuestSummary() is a read-only overview function. Unlike viewAvailableEvents() which shows free venues, this function shows the opposite — all venues that are currently booked, along with the client name and event date. It gives the admin a quick overview of ongoing process.

The function has two layers of empty-check. The outer check (head == nullptr) catches the case where no venues have been added to it at all. The inner found Any flag catches the case where venues exist but none are currently booked. Both cases result in a descriptive messages rather than simply printing nothing, which confuse's the user.

The output format — Client: [name] | Event ID: [id] | Date: [date] — uses the pipe character as a visual separator to keep all information on a single line per booking. This is a common console UI pattern for compact summary tables.

16. manageEventsMenu() — The Admin Sub-Menu

```
void manageEventsMenu(EventManager& manager) {
    int opt, id;
    do {
        system("cls");
        cout << "\n\t\t--- Admin: Manage Venues ---\n";
        cout << "\n   1. Add New Venue/Event slot";
        cout << "\n   2. Back to Main Menu";
        cout << "\n\n   Select Option: ";
        cin >> opt;
        if (opt == 1) {
            cout << "\n   Enter New Event ID (Number) : ";
            cin >> id;
            EventNode* temp = head;
            bool exists = false;
            while (temp != nullptr) {
                if (temp->eventID == id) exists = true;
                temp = temp->next;
            }
            if (exists) {
                cout << "\n   Error: Event ID already exists.";
                getch();
            } else {
                manager.addEvent(id);
            }
        }
    } while (opt != 2);
}
```

Figure No.16

16.1 Why a Separate Sub-Menu?

The method manageEventsMenu() is not a member of the EventManager class; rather, it's its own separate function. That demonstrates that menu traversal is distinct from the rest of the code, and should not be mixed

up. The operations on data like addEvent(), bookEvent() and others are part of EventManager methods. The menu method controls interface behavior through displaying options and selecting operations.

16.2 Duplicate Event ID Check

Before invoking manager.addEvent(id), the function checks the whole linked list to ensure that there is no node containing an event ID matching the input value. In such a case, it displays an error and returns back to the sub-menu without creating any entry. The reason behind it is to avoid duplication of events and ambiguity while searching for them in bookEvent() and generateBill().

16.3 Pass-by-Reference — EventManager& manager

The function parameter is EventManager& manager — the ampersand means it is a reference, not a copy. This means any method called on manager inside the function actually runs on the same EventManager object that was passed in from main(). While EventManager has no data members of its own (so copying it would not cause real harm here), passing by reference is the correct and efficient pattern for passing objects to functions.

17. main() — Program Entry Point and Menu Controller

```
int main() {
    EventManager manager;
    int choice;
    string searchName;
    int searchID;
    do {
        system("cls");
        cout << "\n\t============================================================";
        cout << "\n\t|                        EVENT MANAGEMENT SYSTEM                        |";
        cout << "\n\t============================================================\n";
        cout << "\n\t   1. Manage Event Venues (Add)";
        cout << "\n\t   2. View Available Venues";
        cout << "\n\t   3. Book an Event";
        cout << "\n\t   4. Search Client Booking";
        cout << "\n\t   5. Checkout & Generate Bill";
```

```
        cout << "\n\t   6. View Guest Summary";
        cout << "\n\t   7. Exit Program";
        cout << "\n\n\t   Enter your choice: ";
        cin >> choice;

        if (choice >= 2 && choice <= 6 && head == nullptr) {
            cout << "\n   Database is empty. Add an Event Venue first.";
            getch(); continue;
        }
        switch (choice) {
            case 1: manageEventsMenu(manager);      break;
            case 2: manager.viewAvailableEvents();  break;
            case 3: manager.bookEvent();            break;
            case 4:
                cout << "\n   Enter Client's First Name: ";
                cin >> searchName;
                manager.searchUserByName(searchName); break;
            case 5:
                cout << "\n   Enter Event ID for Checkout: ";
                cin >> searchID;
                manager.generateBill(searchID); break;
            case 6: manager.viewGuestSummary(); break;
            case 7: cout << "\n   Shutting down. Thank you!\n"; break;
            default: cout << "\n   Invalid Input."; getch(); break;
        }
    } while (choice != 7);
    return 0;
}
```

Figure No.17

17.1 The do-while Loop

main() uses a do-while loop rather than a while loop. The key difference is that do-while always executes its body at least once before checking the exit condition. This is appropriate for a menu-driven program because you always want the menu to appear at least once when the program starts. The loop continues until the user enters 7 (exit), at which point choice == 7 makes the condition false and the loop terminates.

17.2 The Empty List Guard

```
if (choice >= 2 && choice <= 6 && head == nullptr) {
```

This guard checks three conditions simultaneously. If the user chooses any option between 2 and 6 AND the linked list is empty (head is nullptr), it prints a message asking the user to add an event first, then uses continue to skip the rest of the loop body and redraw the menu. Without this guard, calling viewAvailableEvents() or bookEvent() on an empty list would traverse a nullptr head and cause a crash and results in no output.

17.3 The switch-case Router

The switch statement connects each integer choice to the function call. Break at the end of each case stops the program for going forward to the next case. If the input is not between 1 and 7, the default case verifyies it and prints an invalid input message. Before calling their functions, cases 4 and 5 need more information. The main() function reads searchName and searchID and sends them as arguments.

18. Simulated Program Output — Complete Walkthrough

The following section presents the expected terminal output for each major operation of the system, using realistic sample inputs. These outputs represent what a user would actually see in the console window.

18.1 Program Startup — Main Menu

```
================================================
|                EVENT MANAGEMENT SYSTEM                |
================================================

  1. Manage Event Venues (Add)
  2. View Available Venues
  3. Book an Event
  4. Search Client Booking
  5. Checkout & Generate Bill
  6. View Guest Summary
  7. Exit Program

  Enter your choice: _
```

This screen appears every time the loop restarts (after each operation completes and the user presses a key). system("cls") clears the previous screen first, so the menu always appears at the top of a clean terminal.

18.2 Adding a New Venue (Option 1)

The user selects 1 → enters the sub-menu → selects 1 again → enters ID 101:

```
--- Admin: Manage Venues ---

  1. Add New Venue/Event slot
  2. Back to Main Menu

  Select Option: 1

  Enter New Event ID (Number) : 101
  Total Guest Capacity : 80
  Time Limit (in minutes) : 120
  Venue Name (One word, e.g., GrandHall) : RoseGarden

  Event Venue Added Successfully!
```

After pressing a key, the system returns to the sub-menu. The user adds a second venue:

```
  Enter New Event ID (Number) : 55
  Total Guest Capacity : 200
  Time Limit (in minutes) : 240
  Venue Name (One word, e.g., GrandHall) : GrandBallroom

  Event Venue Added Successfully!
```

Internally, node ID=55 is inserted at the head first (giving list: 55→101), then mergeSort() runs and re-orders the list to 55→101 (which happens to already be sorted here). If ID=101 had been entered first and then ID=55, the list would be 101→55 after insertion, and mergeSort() would correct it to 55→101.

18.3 Viewing Available Venues (Option 2)

```
--- Available Venues ---

  Event ID     : 55
  Venue Name   : GrandBallroom
  Max Guests   : 200
  Time Limit   : 240 minutes
  ---------------------------------
  Event ID     : 101
  Venue Name   : RoseGarden
  Max Guests   : 80
  Time Limit   : 120 minutes
  ---------------------------------

  Press any key to return...
```

Both venues appear sorted by Event ID (55 before 101). The list is traversed from head to tail; since mergeSort() has been applied, the output is always in ascending ID order.

18.4 Booking an Event (Option 3)

```
  Enter Event ID to book : 101

--- Event Booking Form ---

  Enter Booking ID Number : 9001
  Enter Client First Name : Harsh
  Enter City            : Delhi
  Enter Phone Number      : 9876543210
  Enter Event Date        : 25-Dec-2024
  Enter Advance Payment   : $1000

  Venue Booked Successfully for Harsh!
```

After this operation, the node for Event ID 101 has isBooked = true and all six client fields populated. It will no longer appear in viewAvailableEvents(). If another user tries to book Event ID 101, they will see:

```
  Sorry, this Event ID is already booked.
```

18.5 Searching a Client (Option 4)

```
  Enter Client's First Name to search: Harsh

--- Client Booking Record ---

  Client Name  : Harsh
  Event ID     : 101
  Venue        : RoseGarden
  Date         : 25-Dec-2024

  Press any key to continue...
```

If the name entered does not match any booked record:

```
  No active bookings found for client: Ahmed
```

18.6 Generating the Final Bill (Option 5)

The user enters Event ID 101. The system finds the booked node and asks for actual figures:

```
  Enter Event ID for Checkout: 101
```

```
--- Final Billing Checkout ---

  Enter Actual Number of Guests : 25
  Enter Actual Time (Minutes)   : 90
```

After calculation, the invoice is displayed:

```
--- Final Invoice ---

  Client Name      : Harsh
  Event ID         : 101
  ----------------------------------------
  Food Cost        : $2500.00
  Staff Cost       : $135.00 (3.00 staff members)
  Fixed Overhead   : $2500.00
  ----------------------------------------
  Grand Total      : $5135.00
  Advance Paid     : $1000.00
  Remaining Due    : $4135.00
  ----------------------------------------

  Enter Bank Account Number to pay remaining balance: 12345678
  Processing Payment... Transaction Successful!

  Event checked out. Venue is now available again.
```

After this, temp->isBooked is set to false. Event ID 101 reappears in viewAvailableEvents() and can be booked by a new client.

18.7 Viewing Guest Summary (Option 6)

If two events are booked — Event ID 55 for client Riya and Event ID 101 for client Harsh:

```
--- All Active Bookings ---

  Client: Riya | Event ID: 55 | Date: 30-Nov-2024
  Client: Harsh | Event ID: 101 | Date: 25-Dec-2024

  Press any key to return...
```

If no venues are booked at all:

```
  No active bookings at the moment.
```

If no venues have been added yet (head is nullptr — caught by the guard in main before reaching this function):

```
  The database is empty. Please Add an Event Venue first (Option 1).
```

18.8 Attempting Options 2-6 on Empty List

```
The database is empty. Please Add an Event Venue first (Option 1).
```

This message is printed by the guard in main() and uses the continue keyword to skip the switch-case entirely, looping back to redraw the menu.

18.9 Exit (Option 7)

```
Shutting down system. Thank you!
```

The do-while condition choice != 7 becomes false, the loop exits, return 0 is reached, and the operating system reclaims all memory that was in use.

19. OOP Concepts Demonstrated in This Program

OOP Concept	Definition	Where It Appears in This Code
Encapsulation	Bundling related data and functions into a single unit	User class groups all client fields; EventManager groups all operations; EventNode groups all venue fields
Abstraction	Hiding internal complexity behind a clean interface	main() calls manager.bookEvent() without knowing how the list is traversed internally
Constructors	Special methods that auto-initialize an object at creation	EventNode() constructor sets next=nullptr and isBooked=false every time a node is created

Scope Resolution Operator ::	Used to define class methods outside the class body	void EventManager::addEvent(int id) — method defined outside the class declaration
Pass by Reference	Passing a variable's address so the function can modify it	mergeSort(EventNode** headRef) modifies the actual head pointer; manageEventsMenu(EventManager& manager) passes the object by reference
Separation of Concerns	Keeping different responsibilities in different code units	Data layer (User, EventNode), algorithm layer (3 sort functions), application layer (EventManager, main)
Nested Objects	Embedding one class object inside another	User client is a field inside EventNode — client data lives directly inside the venue node

Table No.10

20. Testing and Validation

Since this is a console application, testing was done manually by running predefined scenarios and checking whether the console output matched the expected result. Six categories of test were performed.

Test ID	Scenario	Input	Expected	Result
T-01	Add event, verify sort	Add IDs 101, 55, 200 in that order	viewAvailable shows: 55, 101, 200	Pass
T-02	Duplicate ID rejected	Add ID 101 twice	Error message on 2nd attempt	Pass
T-03	Book available event	Book ID 55 with name Riya	isBooked=true, fields stored	Pass

T-04	Book already-booked event	Book ID 55 again	Already booked error	Pass
T-05	Bill math verification	25 guests, 90 mins, $1000 advance	Grand: $5135, Due: $4135	Pass
T-06	Search existing client	Search 'Riya'	Record displayed	Pass
T-07	Search non-existent client	Search 'Ahmed'	Not found message	Pass
T-08	View summary with no bookings	Call option 6 when no booking	No active bookings message	Pass
T-09	Operations on empty list	Try option 2-6 before adding	Empty database guard triggers	Pass
T-10	Venue freed after bill	Generate bill for ID 55	ID 55 appears in view after checkout	Pass

Table No. 11

21. Limitations and Future Scope

21.1 Known Limitations:

- No file storage: all data is held in memory. When the program closes, every record disappears. A fresh run starts with an empty list.
- Input validation is minimal: typing a letter when a number is expected (e.g., entering 'abc' for Event ID) causes cin to fail and may produce erratic behavior in the menu loop.

21.2 Future Enhancements:

- Data validation: wrapping all cin commands in while(!cin) loops with cin.clear() and cin.ignore() would increase security for the code by handling bad inputs.
- Platform independence: replacing getch() command with a portable solution would make code more cross-platform.

- Variety of clients at one venue: adapting the system to store multiple User objects for each time slot, instead of only one, would produce a more practical program.

22. Conclusion

In this project, we show that programming constructs like data structures, pointers, dynamic memory allocation, linked lists, and sorting are not just theoretical constructs taught in the first-year computer science course. With the proper implementation, these concepts offer a solution to practical issues.

Firstly, we started off by defining an Event Management System as a problem of data management since the problem involved creation and deletion of data dynamically at runtime. It was imperative that we use the concept of a singly linked list as opposed to arrays, considering that we do not know how many venues will be added before compilation. Every call to addEvent() creates a new pointer through new, and this memory lasts precisely until it is required.

23. References

1. Stroustrup, B. (2013). The C++ Programming Language (4th Edition). Addison-Wesley Professional.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd Edition). MIT Press. [Chapters 2 and 10]
3. Sedgewick, R., & Wayne, K. (2011). Algorithms (4th Edition). Addison-Wesley. [Chapter 2 — Sorting]
4. cppreference.com — C++ Standard Library Reference. Retrieved 2024. https://en.cppreference.com
5. GeeksforGeeks — Merge Sort for Linked Lists. Retrieved 2024. https://www.geeksforgeeks.org/merge-sort-for-linked-list/
6. Class Lecture Notes — Object-Oriented Programming in C++, B.Tech Semester II, 2026.